Azure Cloud Fundamentals and Data Pipeline Implementation using ADF

Week 4 – Assignment 4

Name	Subhodeep Samanta
Institution	DIT University
Email	1000020057@dit.edu.in

Azure Subscription	Azure for Students
Region Used	Malaysia West
Dataset	Sample Superstore dataset (Kaggle)

Objective

The goal of this assignment was to get hands-on with the basics of Azure and build a working data pipeline using Azure Data Factory (ADF). I hadn't used ADF much before this, so most of the time went into exploring the portal, understanding how Storage Accounts, Linked Services, Datasets and Pipelines connect to each other, and then actually getting a Copy Data pipeline to run end-to-end on a real dataset. I used the Sample Superstore dataset from Kaggle as the source file for the pipeline, since it's a reasonably sized CSV that's good for testing things like schema detection and the Get Metadata activity.

Below I've documented each task in the order I did them, along with the screenshots I took at each step and a short note on what happened - including a couple of errors I ran into and how I fixed them, since that ended up being a useful part of actually understanding how ADF works.

Task 1: Exploring the Azure Portal & Creating a Resource Group

I started by logging into the Azure Portal with my Azure for Students subscription. Before creating anything, I spent a few minutes just clicking around the portal home page to get a feel for where things are - the resource groups view, the subscriptions section, and the search bar at the top which turned out to be the fastest way to find any service instead of digging through menus.

For the actual task, I created a new Resource Group called **rg-superstore-pipeline**. A resource group is basically a container that holds all the related resources for a project - so instead of my storage account, data factory, etc. being scattered around, they all sit inside this one group, which makes it much easier to manage and eventually clean up. I set the region to **Malaysia West** since that was the closest available region for my subscription, and kept every resource after this in the same region so latency and billing stay consistent.

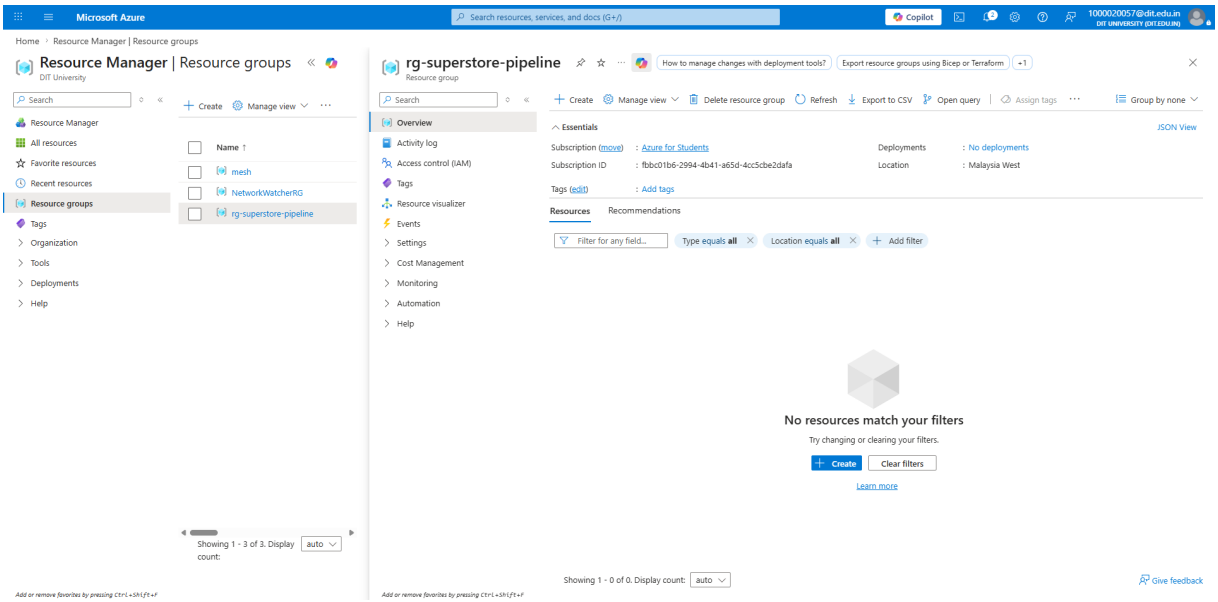

Fig 1: Resource group rg-superstore-pipeline created successfully - Overview page showing subscription, region and no deployed resources yet.

Task 2: Storage Account, Blob Container & Uploading the CSV

Next, inside the same resource group, I created a Storage Account named **stsuperstoreadf**. Storage account names have to be globally unique across all of Azure and lowercase-only, so I had to think of something specific enough not to clash with someone else's. I kept the performance tier as Standard and redundancy as LRS (locally redundant storage) since this is just for learning purposes and doesn't need geo-replication.

Once the storage account was up, I went into **Containers** under Data storage and created a new blob container called **superstore-container**, keeping the access level Private so it's not publicly exposed. I then downloaded the Superstore dataset from Kaggle and uploaded the CSV directly into this container using the Upload button. The screenshot below confirms the file landed correctly - it shows up as a Block Blob, about 2.18 MiB in size.

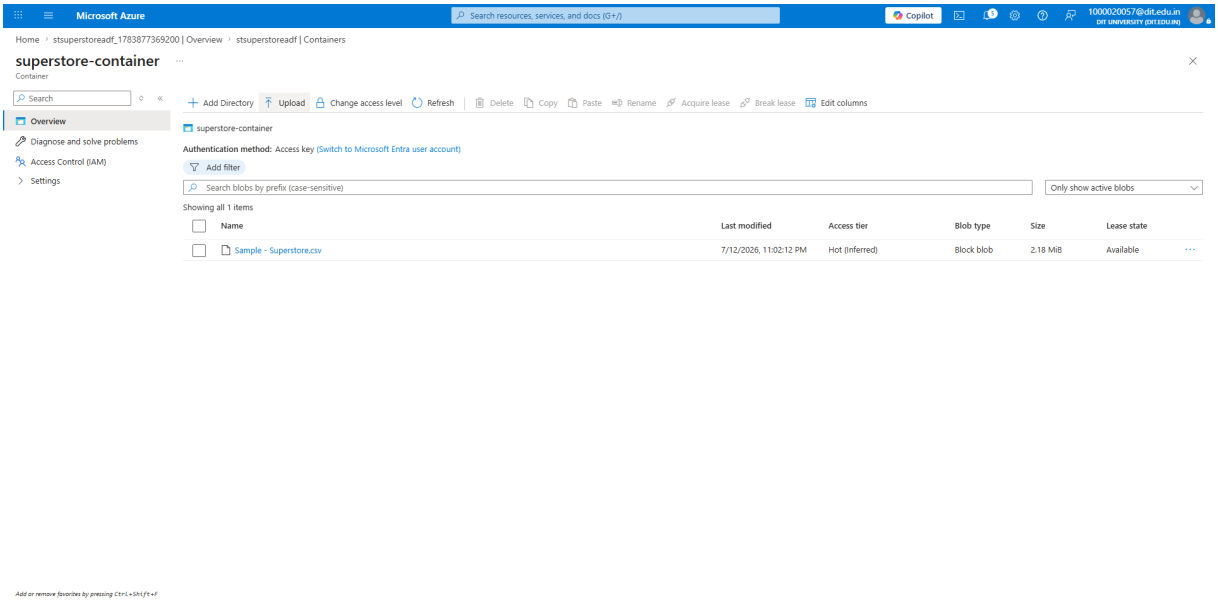

Fig 2: superstore-container showing the uploaded Sample - Superstore.csv file (Block blob, 2.18 MiB).

Task 3: Setting Up Azure Data Factory

This is where most of the actual learning happened. I created an Azure Data Factory instance called **adf-superstore-pipelineSS** inside the same resource group and region. I chose ADF Version 2 since that's the current version with the visual pipeline designer, and for Git configuration I chose "Configure Git later" since I didn't need source control integration for this assignment.

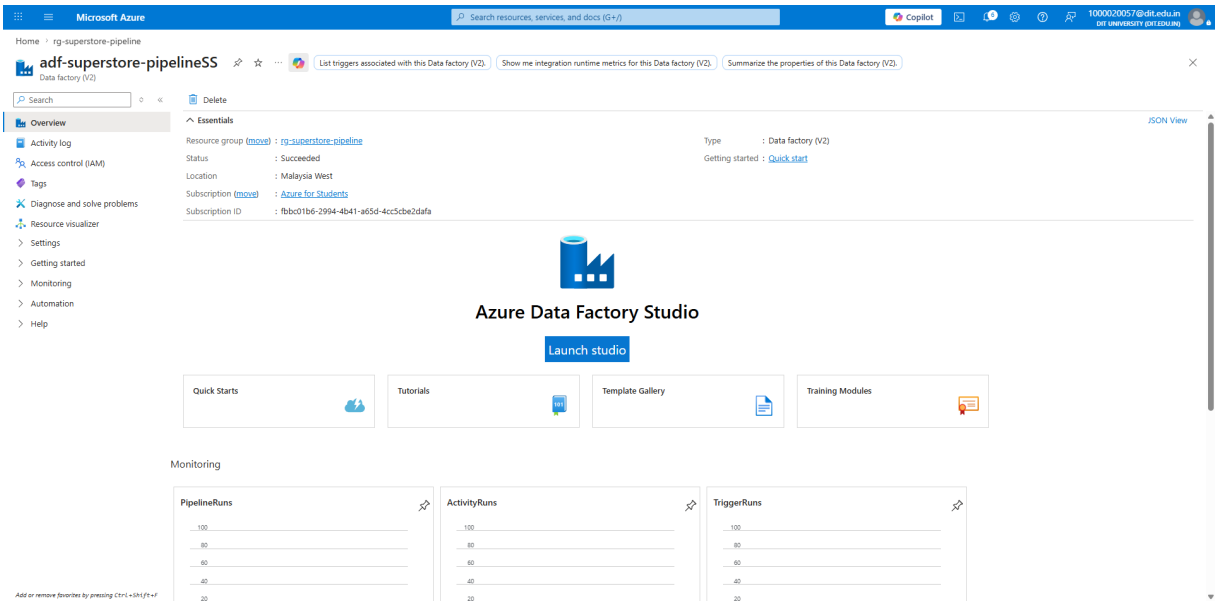

Fig 3: Data Factory (V2) resource created - Overview page showing status Succeeded, resource group and region.

Exploring the ADF Studio UI

From the Overview page I clicked **Launch Studio**, which opens the actual ADF authoring interface (this is separate from the regular Azure Portal, and runs at adf.azure.com). The left sidebar has a few key sections I spent some time understanding:

• **Author** (pencil icon) - this is where you actually build things: pipelines, datasets, data flows, etc.
• **Monitor** (dial/gauge icon) - shows the history and status of pipeline runs, so you can see whether something succeeded or failed and drill into why.
• **Manage** (toolbox icon) - this is where the "plumbing" lives: linked services, integration runtimes, triggers, and security/credentials settings.

Creating the Linked Service

Under Manage → Linked services, I created a new linked service of type **Azure Blob Storage** and named it **ls_blob_superstore**. A linked service is essentially the connection string / credential ADF uses to talk to an external resource - in this case, my storage account. I used Account key authentication and selected my storage account (stsuperstoreadf) directly from my subscription. Running Test Connection came back with "Connection successful," confirming ADF could actually reach the storage account before I built anything on top of it.

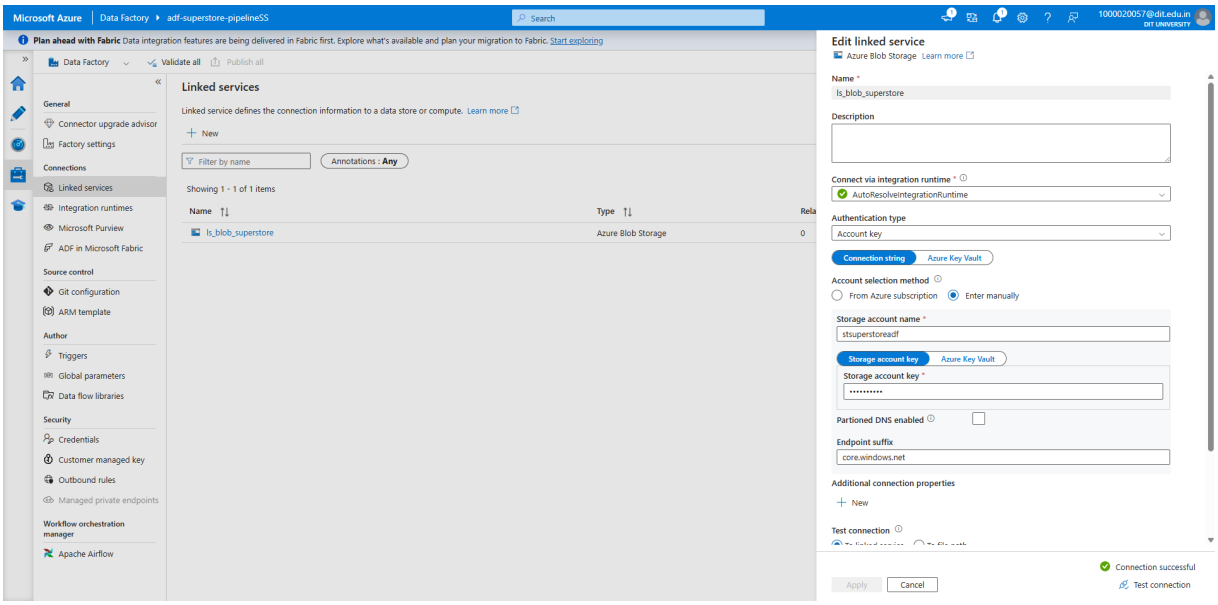

Fig 4: Linked service ls_blob_superstore configured with Account key authentication - test connection successful.

Creating the Source and Destination Datasets

With the linked service in place, I created two datasets under Author → Datasets:

• **ds_superstore_source** - pointing at the uploaded CSV inside superstore-container, with "First row as header" checked and schema imported from the actual file.

• **ds_superstore_destination** - pointing at a new path, superstore-container/output/superstore_copy.csv, which is where the pipeline would eventually write the copied file.

One thing that tripped me up here: when I tried to import the schema for the destination dataset, ADF threw a "Schema import failed - the required Blob is missing" / 404 error. At first I thought I'd misconfigured something, but it actually made sense once I thought about it - the destination file doesn't exist yet at that point, since it only gets created once the pipeline actually runs. The fix was simple: I changed the "Import schema" option for the destination dataset to **None** instead of "From connection/store," which is the normal thing to do for sink/destination datasets anyway.

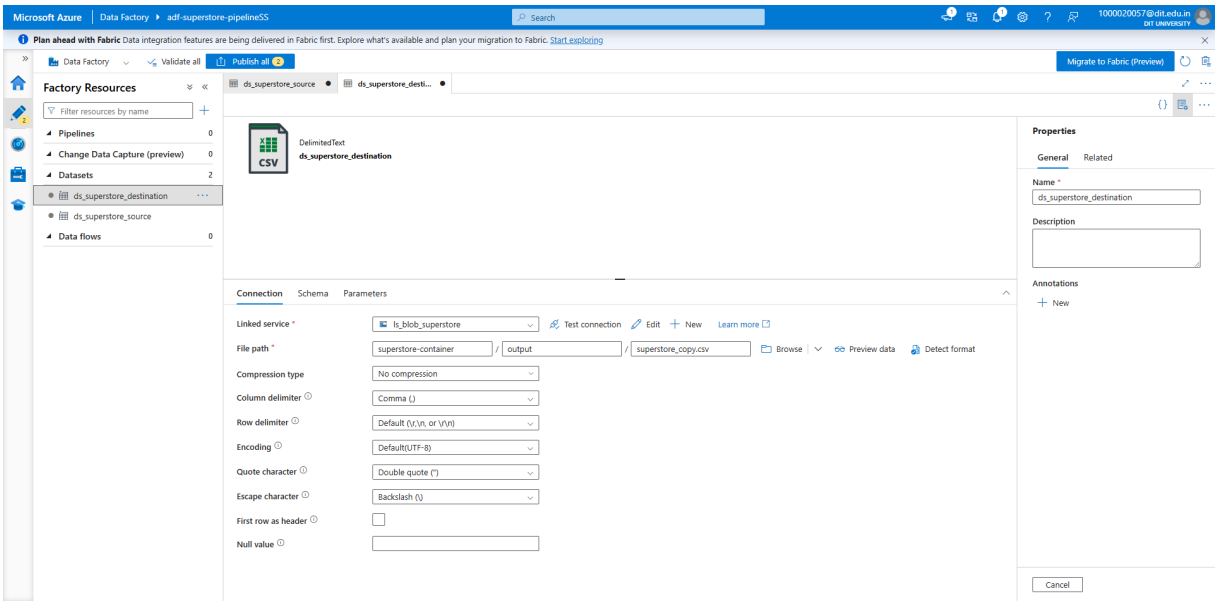

Fig 5: Destination dataset ds_superstore_destination - connection settings showing the output file path, delimiter and encoding configuration.

Get Metadata Activity

To get familiar with metadata retrieval before building the full copy pipeline, I created a small test pipeline called **pl_get_metadata_check** with just a Get Metadata activity pointed at the source dataset. I ran it using Debug, and it completed successfully in about 17 seconds, confirming ADF could correctly read file-level metadata (like Item name, Size, Last modified) from the CSV before I moved on to the actual Copy Data pipeline.

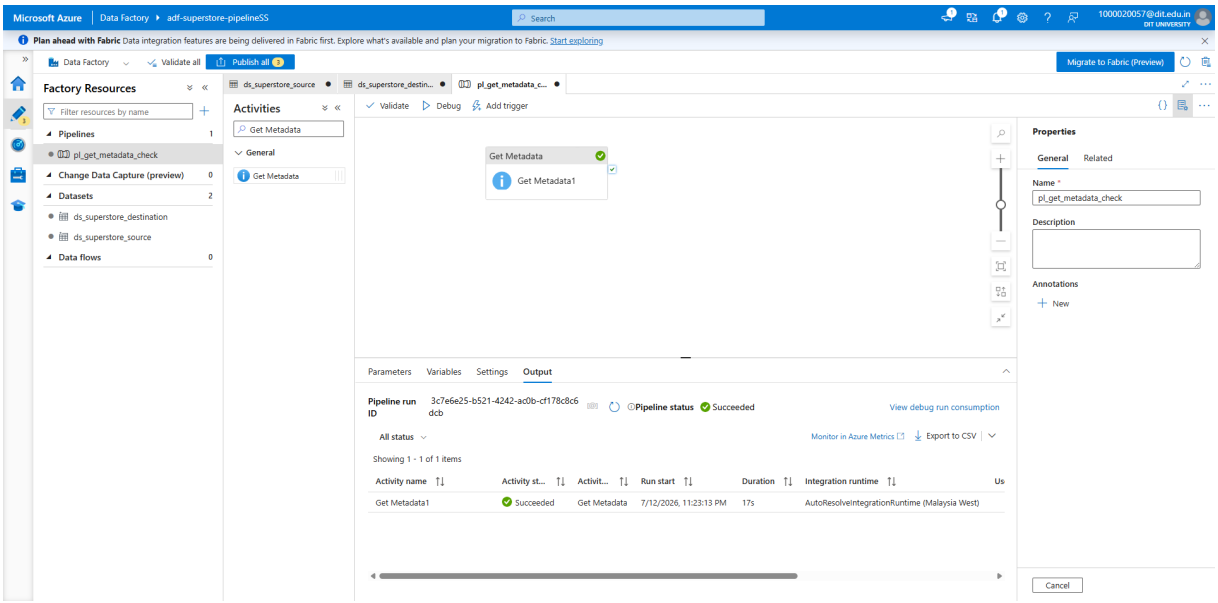

Fig 6: pl_get_metadata_check pipeline - Debug run output showing Pipeline status Succeeded.

Task 4: Building the Copy Data Pipeline

With the datasets and linked service tested individually, I created the main pipeline, **pl_copy_superstore_data**, and added a **Copy data** activity to the canvas. In the Source tab I selected ds_superstore_source and left the file path type as "File path in dataset" since I was only working with a single file - a ForEach loop wasn't necessary here since there was no need to iterate over multiple files. In the Sink tab I pointed it at ds_superstore_destination, so the activity's job was simply to copy the Superstore CSV from its original location to the output/ subfolder in the same container.

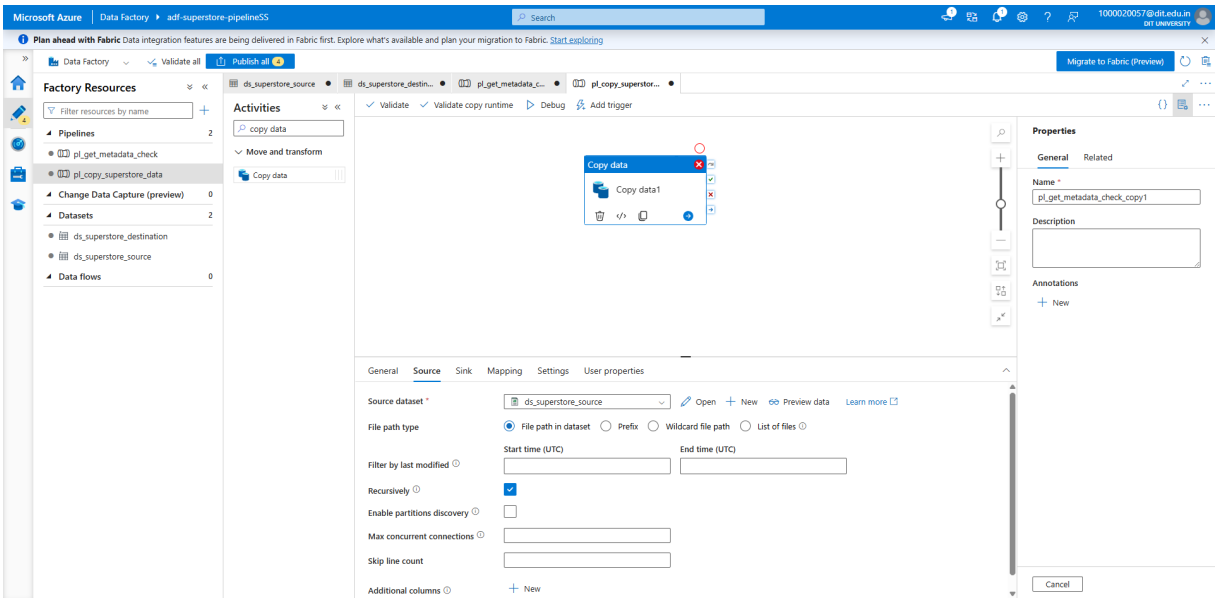

Fig 7: pl_copy_superstore_data pipeline canvas with the Copy data activity - Source tab showing ds_superstore_source configuration.

After first testing this pipeline with just the Copy data activity on its own, I went back and added a **Get Metadata** activity directly onto this same canvas, placed before Copy data and connected to it with an "On Success" dependency (the green arrow). This way the pipeline checks the source file's metadata first and only proceeds to copy it once that check succeeds - which is the real end-to-end behaviour the mini project below is describing, combined into a single pipeline rather than split across two.

Task 5: Running the Pipeline

When I first ran this pipeline using Debug, it actually failed with a **DelimitedTextMoreColumnsThanDefined** error on row 34 of the source file - the parser detected more columns in that row than the schema expected. This turned out to be a real data-quality quirk in the Superstore CSV itself: one of the text fields on that row contains a comma inside a value (for instance an address or product name written without proper quote-wrapping), which throws off the column count. I resolved it by clearing the fixed column mapping on the Copy activity so it would map columns dynamically at runtime instead of enforcing a strict 21-column schema, and re-ran Debug.

After that fix, the pipeline ran successfully. The Monitor/Output view showed the Copy data1 activity with status **Succeeded** in 17 seconds, and checking the storage container afterwards confirmed the copied file existed at the destination path.

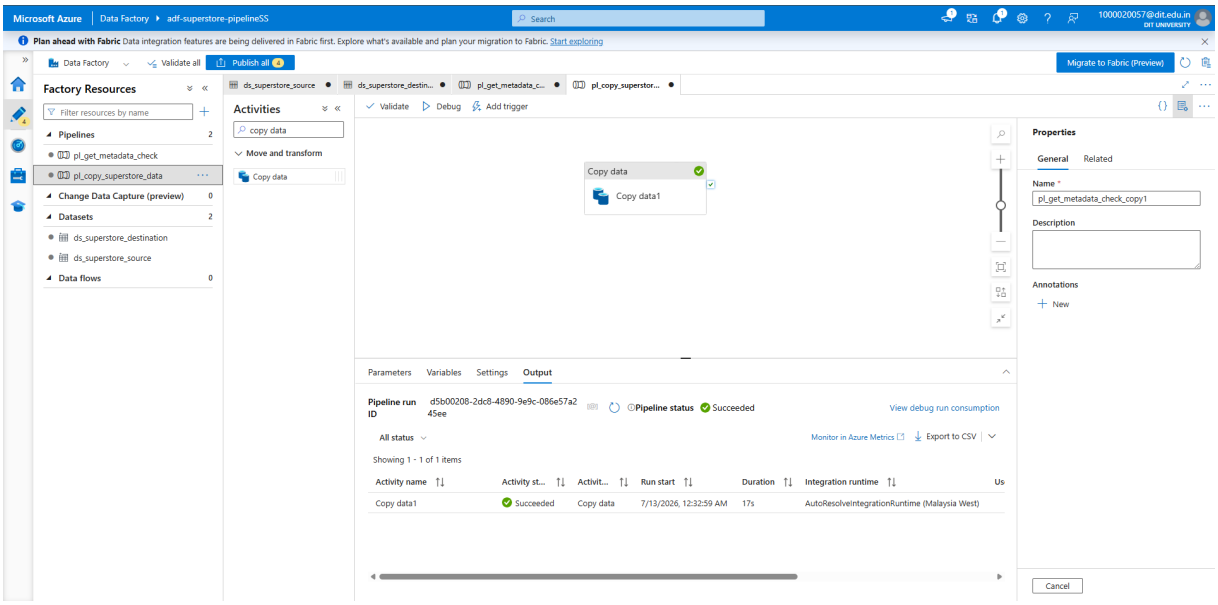

Fig 8: pl_copy_superstore_data - Debug run output confirming Pipeline status Succeeded (Copy data1 activity completed in 17s).

Task 6: IAM Role Assignments

The last configuration task was setting up access control on the storage account so that both I and the Data Factory itself had the right level of access. I went to the storage account's **Access control (IAM)** blade and assigned two standard roles to my own account:

• **Reader** - view-only access, useful for anyone who just needs to check the resource without being able to change anything.
• **Contributor** - full management access to the storage account (create/modify/delete resources) without being able to change access permissions for others.

I also assigned **Storage Blob Data Contributor** to the Data Factory's managed identity itself (rather than to a user account). This is the role that actually governs whether ADF can read and write blob data at the data-plane level - the account-key authentication I used earlier for the linked service works independently of this, but setting up the managed identity role as well reflects the more secure, production-style way of giving ADF storage access without relying on a shared key.

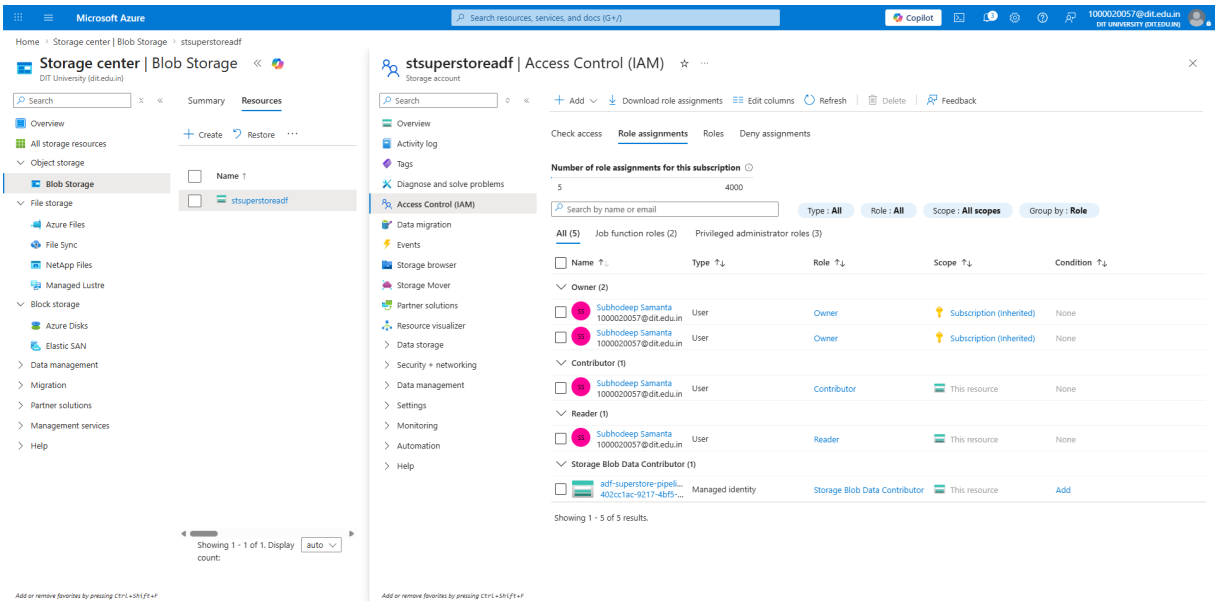

Fig 9: Access control (IAM) - Role assignments on stsuperstoreadf showing Owner (inherited), Contributor, Reader for my account, and Storage Blob Data Contributor assigned to the ADF managed identity.

Mini Project: End-to-End Pipeline Summary

The brief for the mini project asks for a single pipeline that both validates a file's metadata and copies it, so once Task 5 confirmed the Copy Data activity worked on its own, I went back into **pl_copy_superstore_data** and added the Get Metadata activity into the same pipeline (rather than leaving it in the separate pl_get_metadata_check pipeline I'd used earlier just to learn how the activity worked). The two activities are chained with an On Success dependency, so the pipeline now genuinely does: check the source file's metadata → if that succeeds, copy the file to the destination.

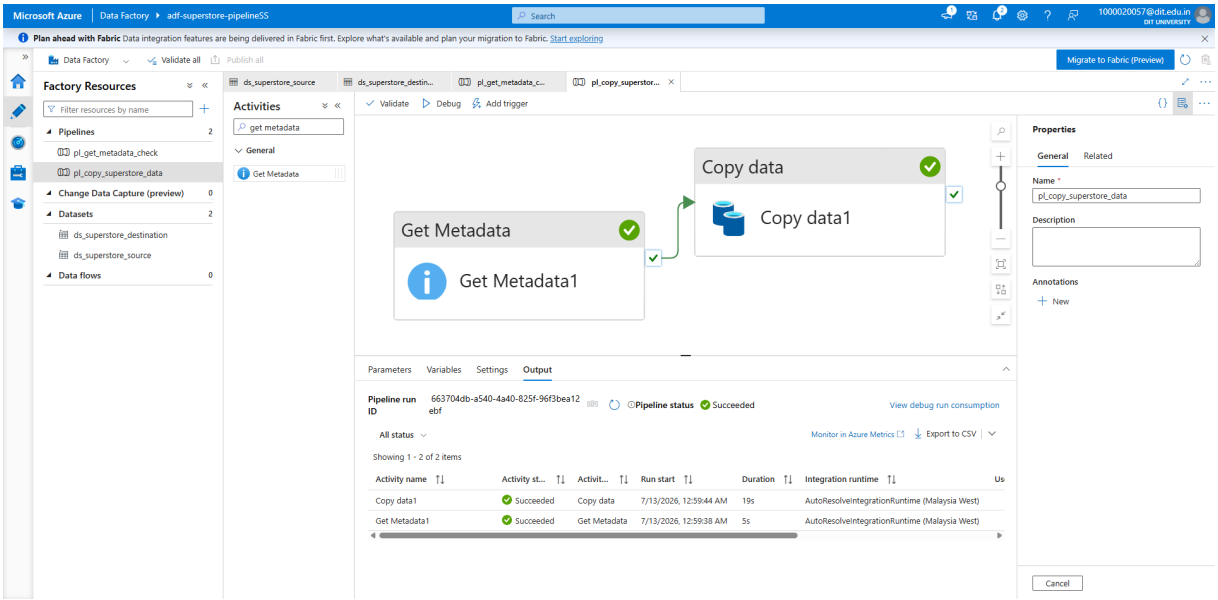

Fig 10: pl_copy_superstore_data with Get Metadata chained to Copy Data via an On Success dependency - Debug output showing both activities Succeeded in a single run (Get Metadata1: 5s, Copy data1: 19s).

Pulling everything together, the final working pipeline looks like this:

Component	Detail

Source	Sample - Superstore.csv in superstore-container (Blob Storage)
Linked Service	ls_blob_superstore (Account key authentication)
Datasets	ds_superstore_source, ds_superstore_destination
Pipeline	pl_copy_superstore_data - Get Metadata → Copy Data (single pipeline, On Success depend
Metadata Check	Get Metadata1 activity, validates source file before copy proceeds
Destination	superstore-container/output/superstore_copy.csv
Result	Single Debug run Succeeded end-to-end; file confirmed present at destination

Overall, the exercise that stuck with me the most was the schema-import error on the destination dataset and the row-34 column-mismatch error on the Copy activity - both were things I wouldn't have understood just from reading documentation. Having to actually debug why ADF was complaining, and realizing one was about a file not existing yet and the other was a genuine data quality issue in the source CSV, made the difference between "following steps" and actually understanding how the pieces of ADF fit together. Restructuring the last part into a single chained pipeline also made it click why dependency arrows (On Success / On Failure) matter in ADF - they're what turn a set of separate activities into an actual controlled workflow.

Conclusion

By the end of this assignment I had a working, end-to-end pipeline in Azure Data Factory that reads a CSV from Blob Storage, validates its metadata, and copies it to a new location - along with a basic understanding of resource groups, storage accounts, linked services, datasets, and IAM role assignments. The biggest takeaway for me was how much of "cloud data engineering" is really about correctly wiring together small, individually simple components (a linked service, a dataset, an activity) rather than any one part being complicated on its own.